Forth on COR24

Four Forth kernels for a 24-bit soft CPU, from all assembly to a dictionary compiled by Forth itself

```
: SQUARE  DUP * ;
  ok
5 SQUARE .
25  ok
```

Mike Wright

2026 October 24 ·sw-embed.github.io/web-sw-cor24-forth

What this deck covers

1. **The target**

   COR24 in one page, and what its ISA takes away from a Forth writer.

2. **Four implementations**

   forth.s, forth-in-forth, forth-on-forthish and forth-from-forth: what each keeps in assembly. Plus forth.s inside SWTOS.

3. **Choices vs. other Forths**

   Threading, cell size, HERE semantics, division, dictionary search.

4. **The demos**

   What each dropdown entry teaches, which tabs run it, and the hardware demos.

5. **Working with words**

   Define, run, list, inspect, redefine and forget, in the web UI and on the CLI.

The target: COR24

A 24-bit RISC soft CPU for Lattice MachXO FPGAs (makerlisp.com)

COR24 at a glance

24-bit

registers, addresses and cells; one cell is 3 bytes

32

opcodes in 211 forms; instructions are 1, 2 or 4 bytes

1

condition flag (C), set by ceq / clu / cls, tested by brt / brf

Register	Role in the Forth kernels
r0	W: work register, holds the fetched CFA
r1	RSP: return stack pointer, SRAM 0x0F0000 growing down
r2	IP: threaded-code instruction pointer
sp	DSP: data stack, hardware push / pop into 3 KB EBR, starts 0xFEEC00
fp	scratch; the only register that can index the EBR stack
z, iv, ir	hardwired zero; interrupt vector and return

Address	Device
0x000000	1 MB SRAM: kernel, dictionary, return stack
0xFEE000	3 KB EBR block RAM: the hardware data stack
0xFF0000	LED D2 (write) and switch S2 (read), same pin, active-low
0xFF0100 / 01	UART data and status (bit 0 RX ready, bit 7 TX busy)

What the ISA takes away from a Forth writer

r — Only r0, r1, r2 take results

Loads and ALU results never land in fp or sp. There is no mov r0, sp: reading the stack pointer is mov fp, sp; push fp; pop r0. DEPTH and .S pay that dance.

± — Branches reach ±127 bytes

Anything farther is la r0, label; jmp (r0): 5 bytes and a clobbered W. The colon-word entry template therefore comes in a 3-byte near and a 6-byte far form.

r1 — jal links through r1

r1 is the return stack pointer, so the kernels never use call / return. Every primitive ends in an inlined NEXT and all nesting goes through the Forth return stack.

÷ — mul, but no divide

/MOD is repeated subtraction: assembly on forth.s, a Forth loop on forth-on-forthish. Printing a number with . is the most expensive thing a demo does.

8 — 8-bit immediates, decimal only

la costs 4 bytes and is the only way to reach I/O: la r0, -65280 for the UART. No string literals, no .align; names are .byte lists padded by hand.

! — A 3 KB data stack that halts

About 1,000 cells. The emulator stops the CPU on overflow, so a word that leaks one cell per loop turn kills a demo after ~500 turns. That exact bug shipped once (see Demos).

Design shared by all four kernels

Direct-threaded code: NEXT is 3 instructions, 5 bytes, inlined at the tail of every primitive

```
lw  r0, 0(r2)    ; W  = mem[IP]  (fetch CFA)
add r2, 3        ; IP += one cell
jmp (r0)         ; run the primitive
```

Dictionary entry: a singly linked list, newest first

```
link      3 bytes  previous entry, 0 ends the chain
flags_len 1 byte   bit7 IMMEDIATE, bit6 HIDDEN, len
name      N bytes  no terminator
CFA       3 or 6 B near/far DOCOL template,
                   or the primitive's own code
```

- Registers are frozen across kernels: r0 = W, r1 = RSP in SRAM, r2 = IP, sp = DSP in EBR.
- HERE, LATEST, STATE and BASE push the variable's address, so Forth code writes HERE @. Unusual (JonesForth does it, ANS does not) but it keeps ! writes trivial.
- No loader: the kernel boots straight into QUIT and everything else, including core/*.fth, arrives as UART text, exactly as if typed.
- The outer loop echoes ok after every clean line and ? after an unknown token; a stack underflow prints a message and returns to the prompt.
- FIND walks the link chain on forth.s; tabs 2 and 3 add a 256-bucket first-character hash in front of the walk.

Choices compared with other popular Forths

Aspect	COR24 Forth	JonesForth	eForth	gforth
Threading	DTC, inlined NEXT	ITC	ITC or subroutine	DTC / ITC / token
Cell	24-bit (3 bytes)	32-bit	16 / 32-bit	32 / 64-bit
Kernel source	asm prims + Forth; tab 4 compiled by Forth	x86 asm + Forth	~30 prims + Forth	C + Forth
Standard	pragmatic subset	subset	mostly ANS	Forth-2012
HERE	variable (HERE @)	variable (HERE @)	value	value
Division	repeated subtraction	hardware idiv	Forth-coded UM/MOD	C / hardware
Dictionary lookup	link walk + first-char hash	link walk	link walk, vocabularies	hashed
I/O model	memory-mapped UART, LED, switch	Linux syscalls	serial port	host OS, files
Numbers	signed 24-bit, BASE; no floats	32-bit, no floats	16 / 32-bit	floats, doubles

Deliberately missing: floating point, strings beyond EMIT, blocks, vocabularies, exceptions. Each would cost assembly the board does not need.

Four implementations

in · in-ish · from: how much of the kernel is Forth

One language, four kernels

1 forth.s

All-assembly kernel

3,498
lines of asm

0
lines of Forth core

~65
asm primitives

Tab 1: the debugger. Runs on the FPGA board; reg-rs golden tests.

2 forth-in-forth

Forth in a slimmed asm host

2,720
lines of asm

276
lines in 4 core tiers

50
asm primitives

Tab 2: IF, ., WORDS and SEE are Forth, loaded at boot.

3 forth-on-forthish

Forth on a Forth-ish substrate

2,659
lines of asm

497
lines in 5 core tiers

~22
irreducible primitives

Tab 3: :, WORD, FIND, NUMBER, INTERPRET, QUIT are Forth.

4 forth-from-forth

Kernel compiled by Forth

83
definitions cross-compiled

3,508
lines of kernel.s, all from Forth

~100
instructions to boot

Tab 4 (default): nothing to compile at boot; tab 3 needs 10 M.

Tab 4's kernel.s is generated entirely from Forth, primitives included, and reproduces itself: run on tab 4's own kernel, the compiler emits the same kernel.s byte for byte.

forth.s: the all-assembly kernel (tab 1)

- Generated: forth-core.s (target-independent DTC Forth) + forth-uart.s (UART at 0xFF0100, return stack at 0x0F0000, halting BYE). The layer contract is what lets SWTOS host the same core (next slide).
- Every word is assembly, including IF / THEN, ., WORDS, \ and (. ~65 primitives, 3,879 bytes assembled.
- Richest vocabulary: FORGET, :NONAME, DO / ?DO / LOOP / I / UNLOOP, CONSTANT, VARIABLE, AGAIN / WHILE / REPEAT, DEPTH, .S.
- Fastest: the Fibonacci demo compiles and runs in about 40 M instructions.
- It is what runs on the physical board and what the reg-rs golden-output suite is baselined against, so it changes slowly.
- In the web UI it backs the debugger: step, step-over, breakpoints, registers, both stacks and memory, with the UART pump feeding input between steps.

A primitive, end to end: LED!

```
entry_led_store:
  .word entry_immediate ; link
  .byte 4               ; len
  .byte 76, 69, 68, 33  ; "LED!"
do_led_store:
  add r1, -3
  sw  r2, 0(r1)    ; save IP
  pop r0           ; n
  lcu r2, 1
  and r0, r2       ; low bit
  la  r2, -65536   ; 0xFF0000
  sb  r0, 0(r2)
  lw  r2, 0(r1)
  add r1, 3
  lw  r0, 0(r2)    ; NEXT
  add r2, 3
  jmp (r0)
```

Note the saved IP: even a leaf primitive spills r2 because pop and la need a second register.

The same core inside SWTOS: Forth as a pane

One core, two layers

forth-core.s dictionary, inner + outer interpreter, every primitive; never touches hardware

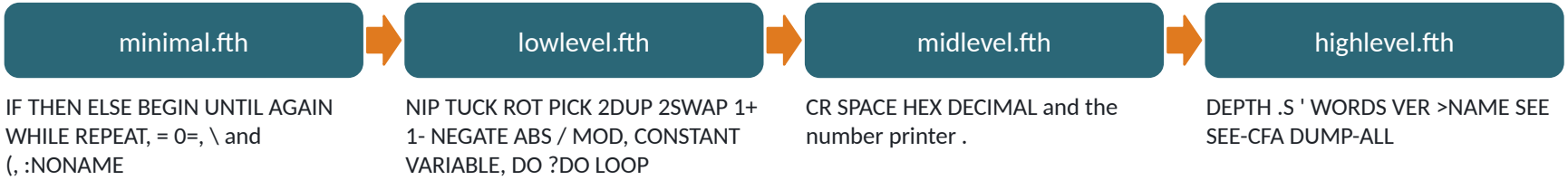
forth-uart.s
UART at 0xFF0100, return stack at 0x0F0000, BYE halts
= forth.s, tab 1 and the board

forth-swtos.s (sw-tos)
I/O via kernel services into its own pane; 8 KB dictionary and 1 KB return stack in the module; BYE or Escape exits the process

```
; the contract, end of forth-core.s
layer_init  io_emit  io_key  io_bye
io_branch  io_zbranch  dict_start  layer_latest
; sw-tos vendors the core with
scripts/vendor-forth-core.sh --check
```

- Live: swtos.softwarewrighter.com/forth/ boots SWTOS's forth profile in the browser. At the shell type bg forth; the REPL opens in its own pane.
- Four layer words: MENU lists the 18 vendored examples, n LOAD runs one as if typed, n LIST prints it, RESET forgets everything. Words are case-insensitive.
- It is a resident program like any other: its data stack is the process stack (256 cells), and a Forth loop that never yields (example 21, HOG) is preempted by the clock and can be killed from the shell. On the bare kernel that loop needs a reset.
- Nothing in the core changed to make this work. The split into forth-core.s plus a layer was done so SWTOS could own I/O, stacks and exit while the Forth stays byte-identical.
- Same source commit as tab 1, so the web debugger tab and the SWTOS pane run the same Forth; only the host differs.

forth-in-forth: Forth extends itself (tab 2)



Each tier uses only primitives and the tiers before it. The kernel streams them in over UART at boot, so WORDS shows them as ordinary colon definitions.

Control flow is four lines of Forth

```
: IF    ['] 0BRANCH , HERE @ 0 , ; IMMEDIATE
: THEN  HERE @ OVER - SWAP ! ; IMMEDIATE
: BEGIN HERE @ ; IMMEDIATE
: UNTIL ['] 0BRANCH , HERE @ - , ; IMMEDIATE
```

- Three new primitives made it possible: ['] (compile a named CFA), EOL! (let a Forth \ end the line) and SP@ (for DEPTH and .S).
- 18 words left assembly; 19 new ones appeared. Kernel 2,983 lines to 2,239 at the time, binary 28% smaller.
- SEE decompiles any colon word, including IF itself.
- Cost: compile-time words run as threaded code. Fibonacci needs ~120 M instructions instead of 40 M; runtime of compiled code is unchanged.

forth-on-forthish: a Forth-shaped substrate (tab 3)

Group	What stays in assembly
Threading	NEXT, DOCOL, EXIT, LIT, BRANCH, 0BRANCH, EXECUTE
Memory	@ ! C@ C!
Arithmetic	+ and NAND (AND, OR, XOR, INVERT, NEGATE, - are derived)
Stack pointers	SP@ SP! RP@ RP! (DUP, SWAP, OVER become memory ops)
I/O	KEY EMIT LED! SW?
Compile	HERE LATEST STATE BASE, ,DOCOL (emits the 6-byte CFA template)
Hand-off	QUIT-VECTOR: the CFA of the Forth QUIT once highlevel.fth has loaded

```
: DUP     SP@ @ ;
: DROP    SP@ 3 + SP! ;
: INVERT  DUP NAND ;
: AND     NAND INVERT ;
: -       NEGATE + ;
: :  CREATE ,DOCOL  ...  ] ;
: ;  ['] EXIT ,  ...  0 STATE ! ; IMMEDIATE
```

- Even : and ; are colon definitions. The only help is ,DOCOL, which copies the COR24 opcode bytes Forth cannot encode itself.
- WORD, FIND, NUMBER, INTERPRET and QUIT are Forth. After the last core line installs QUIT-VECTOR, every prompt line runs through Forth code.
- kernel.s stays at 2,659 lines here: the asm bootstrap must compile runtime.fth before the Forth : exists, so the asm WORD, FIND and NUMBER stay. Tab 4 removes them by compiling the core ahead of time.
- Cost: Fibonacci needs an 800 M instruction budget; stack ops via SP@ are several times slower than push / pop.

forth-from-forth (tab 4): compiled by Forth



What the compiler emits for : R@ RP@ 3 + @ ;

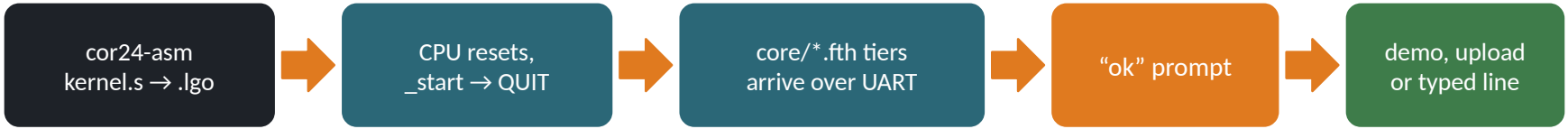
```
fff_e5_R_64:          ; header
.word fff_e4_SWAP     ; link
.byte 2               ; length
.byte 82, 64          ; "R@"
fff_c5_R_64:          ; CFA: far DOCOL
.byte 125, 41 / .word do_docol_far / .byte 38
.word do_rp_fetch     ; body
.word do_lit, 3, do_plus, do_fetch, do_exit
```

Retargeting to another CPU becomes a new assembly emitter, not a new kernel.

Shipped 2026-10-23 / 24

- All five core tiers compile from source: 83 definitions, each identical in SEE form to what tab 3 compiles at boot, and they run when linked.
- Boot to the prompt: about 100 instructions, against 10.2 M for tab 3, which compiles its core from text first.
- Gone: the asm bootstrap (INTERPRET, QUIT, NUMBER, WORD, FIND, : ; [']) and the FIND hash table: 9,450 → 7,098 bytes.
- Unblocked by a one-cell bug: tab 3's Forth NUMBER read the minus sign from the wrong cell, so every -1 in the compiler vanished.
- The primitives are Forth too, and the kernel rebuilds itself to the same bytes, a fixed point; tab 4's Rebuild kernel button does it in the browser.

Boot: a stream of core text, or none at all



Tab 1 has no core files; tab 2 streams four tiers, tab 3 five; tab 4 none, its core is compiled in. CLI:

```
cor24-asm forth-on-forthish/kernel.s -o kernel.lgo
cor24-emu --lgo kernel.lgo --speed 0 -n 800000000 \
  -u "$(cat forth-on-forthish/core/*.fth examples/14-fib.fth)"
```

- In the browser a pump hands bytes to the UART only while the PC sits in one of the kernel's known poll loops, in sub-batches of 2 k instructions, then lets compile work run in 50 k batches.
- Those poll-loop labels are the one thing a new kernel must register; a headless test boots all four kernels through the pump, with a watchdog, to catch a missing one.

Kernel	Boot: real work	Core arrives
forth.s	~100	all assembly
forth-from-forth	~100	compiled into the image
forth-in-forth	1.74 M	4 tiers over the UART
forth-on-forthish	10.2 M	5 tiers over the UART

Trade-offs between the four

	forth.s	forth-in-forth	forth-on-forthish	forth-from-forth
Speed	fastest; native compile and run	runtime same; compile ~3× slower	everything through Forth	as tab 3 per line; boot ~100 instructions
Assembly to maintain	~3,500 lines, every word	~2,700 lines, 50 primitives	~2,650 lines; ~22 matter after boot	none as .s: primitives in a Forth assembler vocabulary
What SEE can show	no SEE	IF, ., WORDS as Forth	also :, WORD, FIND, INTERPRET	same, all compiled ahead of time
Retarget to a new CPU	rewrite everything	50 primitives + bootstrap	~22 primitives + bootstrap	new asm emitter + ~22 primitives
Bootstrap risk	none	tier order matters	asm loop must compile Forth : first	none at run time; the compiler carries it
Best for	the board, debugging, golden tests	learning how Forth defines itself	seeing the interpreter as Forth	fast start; the self-hosting story end to end

The demos

The dropdown in every tab, backed by sw-cor24-forth/examples/*.fth

Language demos: what each one teaches

Demo	File	Teaches	Tabs
Smoke Test	00	.S, arithmetic, HEX, WORDS: interpret mode only	1 2 3 4
Colon Definitions	01	TWO, SQUARE, CUBE: compile mode and nesting	1 2 3 4
Math Words	03	NEGATE, DOUBLE, TRIPLE, *	1 2 3 4
ASCII Stars	04	EMIT patterns from composed words	1 2 3 4
Comments	06	\ line comments and ( ) inline comments	1 2 3 4
Division & Modulo	07	/MOD, then / and MOD built from it	1 2 3 4
IF THEN / IF ELSE	08 09	conditional execution inside a definition	1 2 3 4
Loop	10	BEGIN UNTIL counting up and down	1 2 3 4
FizzBuzz	11	nested IF ELSE inside BEGIN UNTIL, 1 to 20	1 2 3 4
Fibonacci (manual / looped)	14	defines its own NIP TUCK 1+ 1-; FIB(0)..FIB(10) or ..FIB(20)	1 2 3 4
AGAIN / WHILE REPEAT	15 16	BEGIN AGAIN with IF EXIT; test-in-middle TRIANGLE sum	2 3 4
CONSTANT / VARIABLE	17 18	named values; a COUNTER cell with BUMP / RESET / SHOW	2 3 4
DO LOOP	19	DO ?DO I UNLOOP, factorial, early EXIT	2 3 4
SEE (all words)	inline	defines SQUARE and CUBE, then DUMP-ALL decompiles every word	2 3 4
Self Test	12	every kernel word; stack empty at the end; intentional ? ? errors	1 2 3 4

Also in examples/, upload as .fth: 20-redefine (shadowing, FORGET), 21-hog (never yields), 22-factorial (recursion), 23-bases.

Hardware demos: S2 in, D2 out



1 LED Control (02)

: ON 1 LED! ; : OFF 0 LED! ;
Then ON, OFF, BLINK. Watch the red dot. Because the pin is active-low, 0 LED! is the one that lights it.

2 Switch → LED (05)

SW? LED! one shot: click S2 first, then Run. The raw bit is copied straight through, so pressed (0) lights D2.

3 Loop Switch LED (13)

MIRROR polls SW? LED! 16,000 times with two delay loops between reads. Toggle S2 while it runs and D2 follows within a tick.

The regression, and the tests that now guard it

- April 2026: the delay loops leaked two cells per turn; the 3 KB stack overflowed after ~500 turns and the halted CPU looked like a dead LED.
- Web: tests/led_switch_demo.rs boots each kernel headlessly, presses S2 mid-run and asserts D2 follows and the stack does not drift.
- CLI: cor24-emu --switch-at 6000000:on,8000000:off logs [S2 @ N] and [LED @ N] lines for reg-rs golden tests.

Define, run, list, redefine

Define

```
: SQUARE ( n -- n*n )  DUP * ;
  ok
```

Colon opens a definition, semicolon closes it. A word can call itself by name (recursion). Definitions live until Reset.

Run

```
5 SQUARE .
25  ok
```

Type the word and press Enter; ↑ ↓ recall history. Demos: pick one, click Run. Your own file: Upload .fth. Stop pauses; Reset reboots and forgets everything.

List and inspect

```
WORDS          \ newest first
SEE SQUARE     \ tabs 2 and 3: DUP * ;
DUMP-ALL       \ every word with address
1 2 .S  DEPTH .
```

WORDS on every tab; SEE, SEE-CFA, DUMP-ALL and ' need the Forth-defined kernels.

Redefine, edit, forget

```
: GREET  72 EMIT 105 EMIT ;
: TWICE  GREET GREET ;
: GREET  72 EMIT 101 EMIT 121 EMIT ; \ shadows
TWICE          \ old GREET, twice
FORGET GREET   \ forth.s only
```

A new definition sits in front of the old one; earlier callers keep the binding they compiled. To edit: fix the source, then FORGET (forth.s) or Reset and re-run the file.

Known quirks and open items

!

2SWAP and 2OVER are wrong on tabs 2 to 4

1 2 3 4 2SWAP leaves 2 1 4 3, not 3 4 1 2. Found while writing the cross-compiler, which avoids them; not fixed yet. Also: . prints -8388608 as -0.

0

LED! and SW? are raw active-low bits

0 LED! lights D2 and SW? returns 0 while pressed. The Help text still says non-zero on and -1 pressed; one of the two should change.

F

FORGET exists only on forth.s

Tabs 2, 3 and 4 redefine by shadowing; reclaiming dictionary space means Reset.

S

Tab 3 starts with a stray stack cell

Its asm bootstrap prints ? ? ? while loading the core and leaves 3051 behind, so .S shows <4> for three items. Tab 4 has no bootstrap and starts clean.

Try it

sw-embed.github.io/web-sw-cor24-forth (four kernels, tab 4 by default; ? for the guide) · swtos.softwarewrighter.com/forth/ (forth.s as an SWTOS pane: bg forth)

Repositories, all under github.com/sw-embed

Web UI	`web-sw-cor24-forth`	**Kernels and demos**	`sw-cor24-forth`
Emulator and CLI	`sw-cor24-emulator`	**Assembler**	`sw-cor24-x-assembler`
SWTOS and its web demo	`sw-tos, web-sw-tos`	**Board**	`makerlisp.com`